MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Loops, functions, arrays, math functions and plotting

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**

MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Part 8: Loops

Programming · Part 8 of 12

TASK 1

The for loop

A **for** loop repeats commands a fixed number of times. The variable k takes each value of **1:3** in turn, one pass per value.

```
for k = 1:3
    disp(k)
end
```

write a for loop that displays the numbers 1 to 5

TASK 2

Accumulate a result

Create the variable before the loop, then update it on every pass. Here total = total + k adds each k to the running sum, so total ends at 10.

```
total = 0;
for k = 1:4
    total = total + k;
end
total
```

sum the numbers 1 to 10 with a for loop and check that total is 55

TASK 3

Change the step size

The colon operator **start:step:stop** sets the step. A negative step counts down, for example 5:-1:1.

```
>> 1:2:7
ans =
     1     3     5     7
```

display 10:-2:0, then use it as the range of a for loop

TASK 4

The while loop

A **while** loop repeats as long as its condition is true. Use it when you do not know the number of passes. Change something inside the loop, or it never ends. Press **Ctrl+C** to stop it. Here x ends at 64.

```
x = 1;
while x < 50
    x = x*2;
end
x
```

start with n = 5 and subtract 1 in a while loop until n reaches 0

FURTHER PRACTICE

Exit a loop early with break

break leaves the loop at once. **continue** skips the rest of the current pass and moves on to the next one.

```
for k = 1:10
    if k > 3
        break
    end
    disp(k)
end
```

stop a for loop over 1:10 as soon as k is greater than 3

MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Part 9: Functions

Programming · Part 9 of 12

TASK 1

What is a function?

A function takes **inputs**, does a calculation and returns **outputs**. You already call built-in functions such as sqrt and sin. You can also write your own.

```
>> y = sqrt(25)
y =
     5
```

call abs(-4) and max([4 9 2]) and name the input and output of each

TASK 2

Write a function

A function file starts with the keyword **function**, then the output, =, the function name and the inputs in parentheses. Save it with the same name as the function.

```
function y = timesTwo(x)
    y = 2*x;
end
```

create a file named timesTwo.m that contains this function

TASK 3

Call your function

Once timesTwo.m is in the Current Folder, call it like any built-in function: its name, then the input in parentheses.

```
>> timesTwo(4)
ans =
     8
```

call timesTwo with the input 7 and store the result in a variable named r

TASK 4

Several inputs and outputs

Separate the inputs with commas. For more than one output, list the output names in **square brackets**. Each output must be assigned inside the function.

```
function [s, p] = sumProd(a, b)
    s = a + b;
    p = a*b;
end
```

create sumProd.m with two inputs, a and b, and two outputs, s and p

TASK 5

Call it with several outputs

Collect the outputs in square brackets, in the same order. The names you choose outside the function can differ from the names inside it.

```
>> [s, p] = sumProd(3, 4)
s =
     7
p =
    12
```

call sumProd(5, 6) and save the results in variables named total and prod

FURTHER PRACTICE

Variables stay inside

A function has its own workspace. Variables created inside it, such as y in timesTwo, disappear when it finishes and never reach your Workspace.

```
>> timesTwo(4);
>> y
Unrecognized function or
variable 'y'.
```

run timesTwo(4); and check that y is not in the Workspace panel

MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Part 10: Arrays

Programming · Part 10 of 12

TASK 1

Create a row vector

An **array** stores many values in one variable. Put the values inside **square brackets**, separated by spaces or commas, to make a row vector.

```
>> v = [1 2 3 4]
v =
     1     2     3     4
```

create a row vector named v with the values 5, 10, 15 and 20

TASK 2

Create a matrix

A matrix has rows and columns. Inside the brackets, a **semicolon** ends a row. Here the semicolon separates rows, it does not hide output.

```
>> A = [1 2; 3 4]
A =
     1     2
     3     4
```

create a 2-by-3 matrix named B with the rows 1 2 3 and 4 5 6

TASK 3

Evenly spaced values

linspace(a,b,n) creates n evenly spaced values from a to b, including both ends. The colon operator a:step:b gives you control of the step instead.

```
>> linspace(0,10,3)
ans =
     0     5    10
```

create 5 evenly spaced values from 0 to 1 with linspace

TASK 4

Index into a vector

Use parentheses to read one element. MATLAB indexing starts at **1**, so v(1) is the first element. The word **end** means the last element.

```
>> v = [10 20 30];
>> v(2)
ans =
    20
>> v(end)
```

get the first and the last element of v = [10 20 30]

TASK 5

Index into a matrix

For a matrix, give the **row** first and then the **column**: A(2,1) is row 2, column 1. A colon means all: A(1,:) is the whole first row.

```
>> A = [1 2; 3 4];
>> A(2,1)
ans =
     3
>> A(1,:)
```

get the element in row 1, column 2 of A, then the whole second column with A(:,2)

TASK 6

Change and add elements

Assign to an index to change that element. Assigning past the end makes the array longer, for example v(4) = 40 adds a fourth element.

```
>> v = [10 20 30];
>> v(2) = 99
v =
    10    99    30
```

change the last element of v to 0, then add a fourth element with the value 40

TASK 7

Size and length

size returns the number of rows and columns. **length** returns the largest dimension, and **numel** counts all elements.

```
>> A = [1 2; 3 4];
>> size(A)
ans =
     2     2
```

find the size of B and the number of elements in B with numel

TASK 8

Functions on arrays

Many built-in functions work on a whole array. For v = [4 9 2 4]: **sum** is 19, **max** is 9, **min** is 2 and **mean** is 4.75. **median** is the middle value after sorting (4) and **mode** is the most frequent value (4).

```
>> v = [4 9 2 4];
>> mean(v)
ans =
    4.7500
>> mode(v)
ans =
     4
```

calculate the mean, median and mode of v = [4 9 2 4]

FURTHER PRACTICE

Transpose with an apostrophe

An apostrophe ' after an array flips rows and columns. A row vector becomes a column vector, and the other way round.

```
>> v = [1 2 3];
>> v'
ans =
     1
     2
     3
```

turn the row vector v into a column vector and store it in a variable named w

MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Part 11: Math Functions

Programming · Part 11 of 12

TASK 1

Sum of a vector

sum adds up all the elements of a vector.
Here 1 + 2 + 3 + 1.5 - 1.5 - 2 gives 4.

```
>> v = [1 2 3 1.5 -1.5 -2];
>> sum(v)
ans =
     4
```

create w = [2 4 6 8] and use sum to check that the total is 20

TASK 2

Sum of a matrix

For a matrix, **sum** adds each column and returns a row vector of column sums.

```
>> m = [1 2 3; -1.5 1.5 2.5; 4 5 6];
>> sum(m)
ans =
    3.5000    8.5000   11.5000
```

find the column sums of [1 2; 3 4] and check that you get 4 and 6

TASK 3

Square root and e

sqrt(x) returns the square root of x. MATLAB has no built-in constant e: **exp(x)** calculates e^x, so exp(1) returns e.

```
>> sqrt(4)
ans =
     2
>> exp(1)
ans =
    2.7183
```

calculate the square root of 4 with sqrt, then find e with exp(1)

TASK 4

Complex numbers

Write a complex number with the imaginary unit **i**. **real** and **imag** return its two parts.

```
>> z = 3+4i;
>> real(z)
ans =
     3
```

use imag(z) to find the imaginary part and check that it is 4

TASK 5

Magnitude and conjugate

abs(z) is the magnitude, sqrt(3^2 + 4^2).
conj(z) flips the sign of the imaginary part.

```
>> abs(z)
ans =
     5
>> conj(z)
ans =
   3.0000 - 4.0000i
```

find angle(z) in radians, which is atan(4/3), and check that it is 0.9273

TASK 6

Exponential and logarithm

exp(x) is e^x. **log** is the natural logarithm ln(x), **log10** is base 10 and **log2** is base 2.

```
>> log(100)
ans =
    4.6052
>> log10(100)
ans =
     2
```

calculate log2(1024) and check that the answer is 10

TASK 7

Trigonometric functions

sin, **cos** and **tan** expect radians, so multiply degrees by pi/180. **sind** takes degrees directly.

```
>> sin(30*pi/180)
ans =
    0.5000
>> sind(30)
ans =
    0.5000
```

use asind(0.5) to get the angle back, and check that it is 30

FURTHER PRACTICE

Counting with the colon

start:step:stop builds a row of evenly stepped values. i = 0:2:10 counts from 0 to 10 in steps of 2.

```
>> i = 0:2:10
i =
     0     2     4     6     8    10
```

create a row of the even numbers from 2 to 20 with the colon operator

MATLAB ONRAMP - SECTION TWO

MATLAB Fundamentals

Part 12: Plotting and Help

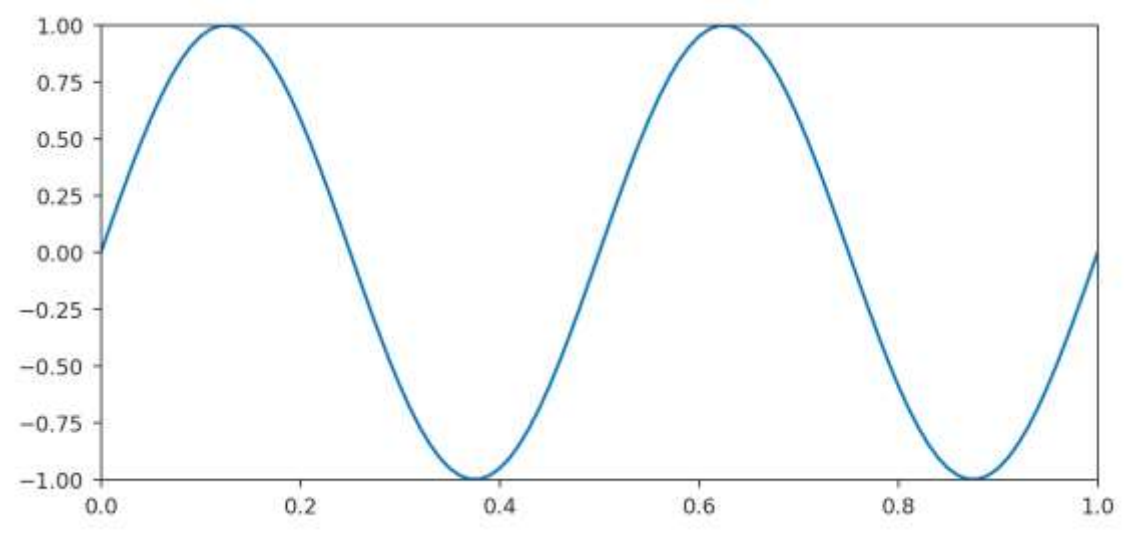
Programming · Part 12 of 12

TASK 1

Plot an analog signal

plot(t,y1) draws an x-y line. Use it for analog, continuous signals.

```
t = 0:.01:1;
y1 = sin(2*pi*2*t);
plot(t,y1)
```



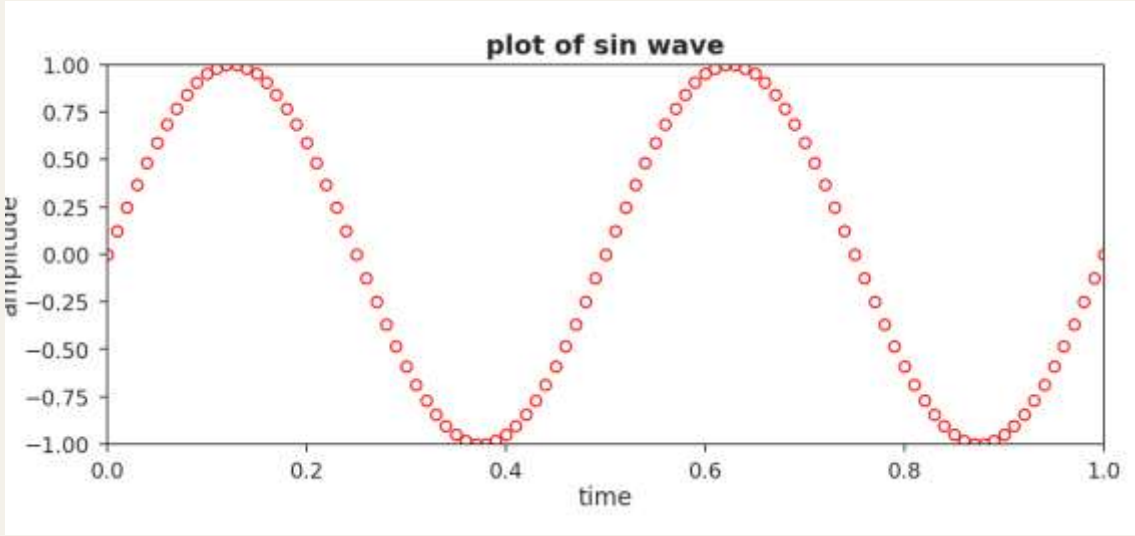
create t and y1 as shown, then draw the sine wave with plot

TASK 2

Colors, markers and labels

'ro' sets red circles. **title**, **xlabel** and **ylabel** label the figure.

```
plot(t,y1,'ro')
title('plot of sin wave')
xlabel('time')
ylabel('amplitude')
```



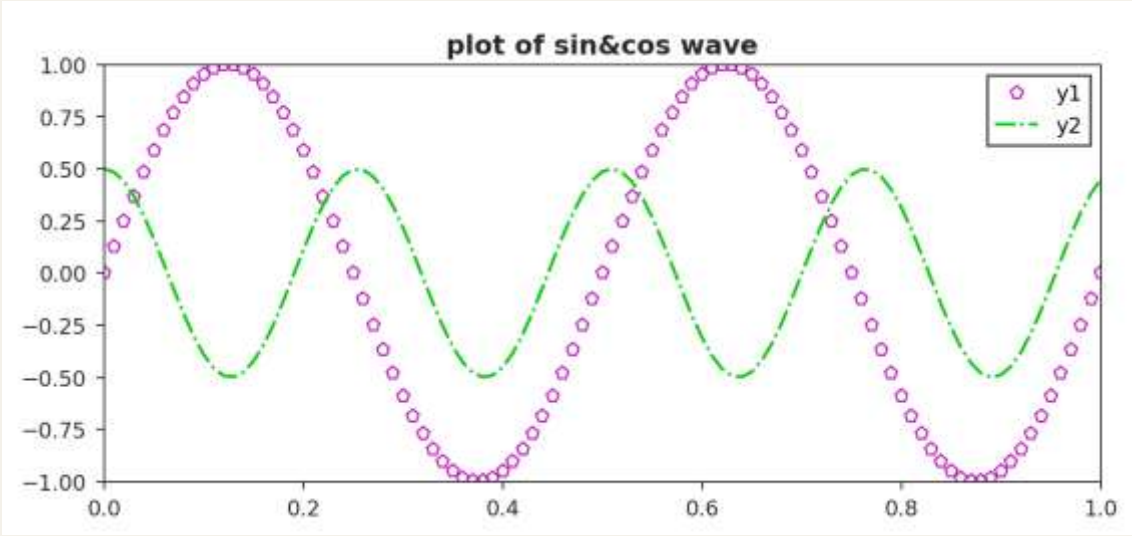
plot y1 again with green stars, using the format 'g*'

TASK 3

Two signals and a legend

Give **plot** several x-y pairs to draw more than one curve, then name them with **legend**.

```
y2 = .5*cos(2*pi*5*t*pi/4);
plot(t,y1,'mp',t,y2,'g-.')
legend('y1','y2')
```



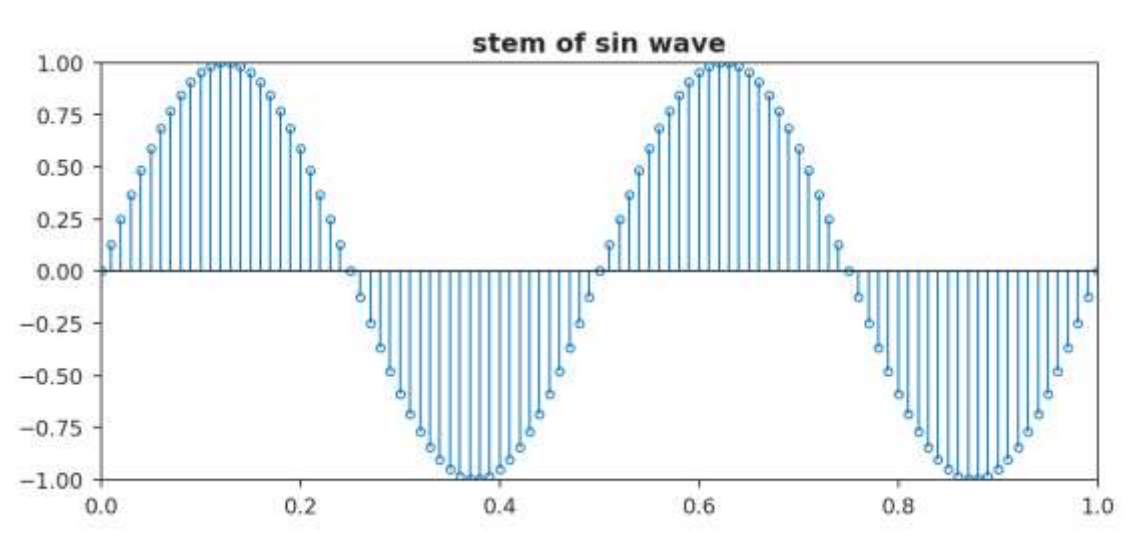
plot y1 and y2 together and add a legend

TASK 4

Stem: discrete-time signals

stem draws each sample as a vertical line with a circle on top. Use it for digital, discrete-time signals.

```
stem(t,y1)
title('stem of sin wave')
```



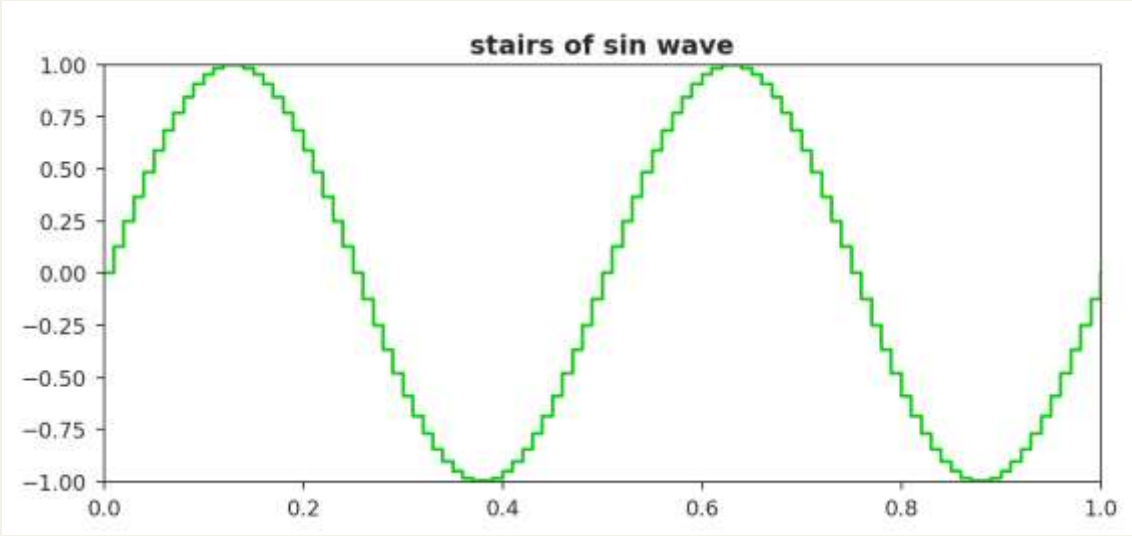
draw stem(t,y1) and compare it with the plot of the same signal

TASK 5

Stairs: sample-and-hold

stairs holds each value until the next sample, which gives a staircase (sample-and-hold) signal.

```
stairs(t,y1,'g-')
title('stairs of sin wave')
```



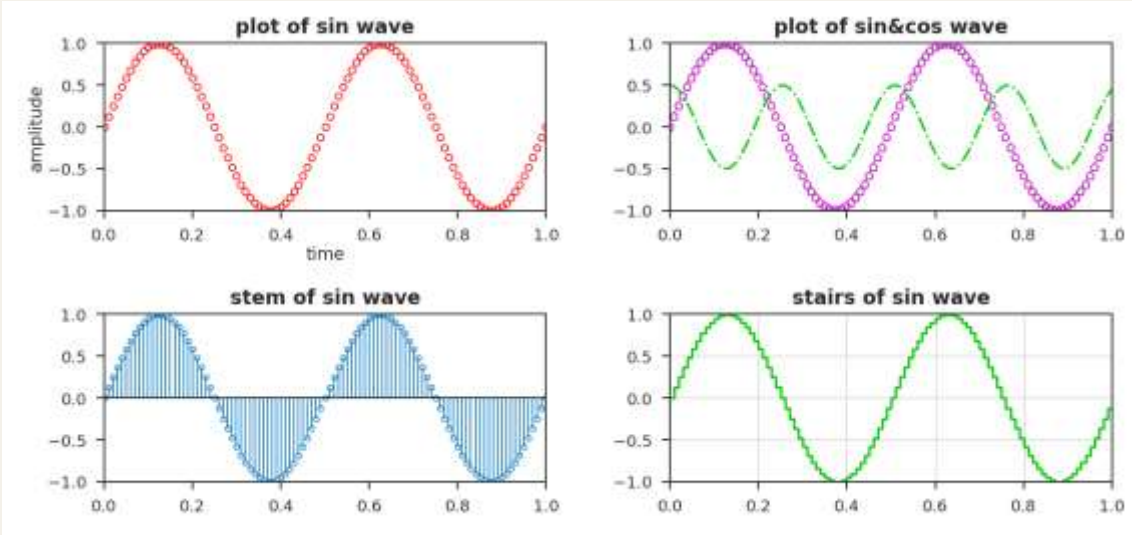
draw the signal with stairs and compare it with stem

TASK 6

Subplot: many plots in one figure

subplot(m,n,p) splits the figure into an m-by-n grid and selects plot p.

```
subplot(2,2,1), plot(t,y1,'ro')
subplot(2,2,2), plot(t,y1,'mp',t,y2,'g-.')
subplot(2,2,3), stem(t,y1)
subplot(2,2,4), stairs(t,y1,'g-')
grid on
```



build the 2-by-2 figure and give each plot its own title

FULL EXAMPLE

Subplot: the full script

```
t = 0:.01:1;
y1 = sin(2*pi*2*t);
y2 = .5*cos(2*pi*5*t*pi/4);
subplot(2,2,1)
plot(t,y1,'ro')
title('plot of sin wave')
xlabel('time')
ylabel('amplitude')
subplot(2,2,2)
plot(t,y1,'mp',t,y2,'g-.')
title('plot of sin&cos wave')
legend('y1','y2')
```

```
subplot(2,2,3)
stem(t,y1)
title('stem of sin wave')
subplot(2,2,4)
stairs(t,y1,'g-')
title('stairs of sin wave')
grid on
```

FURTHER PRACTICE

Get help: help and doc

help shows a short description in the Command Window. **doc** opens the full MATLAB documentation in a separate window, with more detail.

```
>> help sin
sin    Sine of argument in radians.
>> doc sin
```

use help and then doc on the function plot and compare what each shows

MATLAB ONRAMP - SECTION TWO

Thank You

Any questions?

Under the supervision of **DR. Lobna Abouelmagd**

Prepared by **ENG. Moamen Yasser**